AQUA TRACK

Software Requirements Specification (SRS)

QR-Based Laundry Tracking & Workflow Management Web Application

Document Item	Value
Document Type	Software Requirements Specification
Project	Aqua Track
Version	1.0
Status	Development / Academic Project
Primary Platform	Responsive Web Application
Frontend	Next.js + TypeScript + Tailwind CSS + shadcn/ui
Application Backend	Next.js Server Actions / API Routes + service layer
Infrastructure	Supabase
Database	PostgreSQL
Authentication	Supabase Auth
Realtime	Supabase Realtime
Deployment	Vercel + Supabase

Prepared from the Aqua Track Master Specification and the current Aqua Track project structure. The requirements below describe the intended product and the architecture currently implemented around it.

Table of Contents

1. Introduction
2. Overall Description
3. Stakeholders and User Roles
4. System Scope
5. Functional Requirements
6. End-to-End Workflow
7. Business Rules and State Management
8. Data Requirements
9. External Interface Requirements
10. Non-Functional Requirements
11. Security and Authorization
12. System Architecture
13. Current Project Structure
14. Testing and Acceptance Criteria

15. MVP Scope and Out of Scope

16. Deployment and Environment Requirements

17. Risks, Edge Cases and Failure Handling

18. Traceability and Definition of Done

19. Conclusion

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the functional, non-functional, data, security, workflow, architectural and testing requirements for Aqua Track. It is intended to serve as a common reference for development, testing, demonstration and future maintenance.

1.2 Problem Statement

Students who submit laundry currently have limited visibility into its lifecycle. They may not know whether pickup is available, whether their bag has been collected, whether it has reached the washing facility, whether washing has started or completed, whether it is ready for delivery, whether it is out for delivery, or whether it has been delivered.

1.3 Product Objective

Aqua Track connects the physical laundry workflow with a digital tracking system. Each meaningful physical handoff produces a corresponding digital event, allowing students to track their laundry while staff use QR-based operations to manage collection, washing and delivery.

1.4 Core Product Principle

Every meaningful physical event should produce a corresponding digital event. The digital status must reflect the real operational state of the physical laundry.

COLLECTED ↓ RECEIVED_AT_FACILITY ↓ UNDER_WASHING ↓ READY_FOR_DELIVERY ↓ OUT_FOR_DELIVERY ↓ DELIVERED

2. Overall Description

2.1 Product Perspective

Aqua Track is a role-based workflow management web application. The UI is implemented with Next.js and React/TypeScript. Server-side application logic is implemented through Next.js server actions, API routes, feature data-access modules and domain services. Supabase provides authentication, PostgreSQL database services and realtime capabilities.

2.2 Core Value Proposition

Audience	Value
Students	Real-time visibility into the journey of their laundry.
Delivery Staff	QR-based collection and delivery operations with batch visibility.
Washing Staff	Structured receipt, queue and washing workflow.
Admin	Operational oversight, user/bag/facility/order management, complaints and auditability.

2.3 Operating Model

A permanent QR identity is associated with each physical laundry bag. When a student submits a bag, the delivery staff scans the QR code. The system creates or resolves the corresponding laundry order. The order then follows controlled status transitions through collection, facility receipt, washing, ready-for-delivery, delivery and completion.

2.4 Assumptions

- Students primarily observe status and do not perform operational bag scans.
- Delivery staff perform collection and delivery scanning.
- Washing staff operate the washing/facility workflow.
- The system has network access during normal scanning and status updates.
- A physical bag can be reused for multiple laundry orders over time.
- QR codes identify the bag and must not expose sensitive student information.

3. Stakeholders and User Roles

Role	Description	Primary Capabilities
Student / User	Customer who owns and submits laundry.	Authenticate; view profile; view pickup announcements; view active laundry; view details/history/timeline; receive notifications; report problems; communicate with relevant staff.
Delivery Boy	Staff member responsible for pickup and delivery.	Manage pickup sessions; scan bags; collect bags; manage collection progress; create/manage delivery batches; scan for delivery; confirm delivery; handle delivery attempts.
Washing Staff	Staff member operating the washing facility.	Receive batches; scan/identify bags; manage processing queue; start washing; complete washing; move orders toward ready-for-delivery.
Admin	System operator.	Manage users, bags, hostels, facilities, batches, orders, complaints, audit information, pickup sessions and system configuration.

3.1 Role Restrictions

- Students cannot normally modify laundry status or operate staff workflows.
- Users must not access another student's laundry.
- Delivery staff cannot perform arbitrary administrative operations.
- Washing staff access must be restricted to authorized facility operations.
- Admin-only operations must be protected by role authorization.

4. System Scope

4.1 In Scope

- Authentication and role-based access control.

- Student, delivery, washing and admin dashboards.
- Bag registration and QR identity.
- Pickup sessions and pickup announcements.
- QR-based bag collection.
- Laundry order creation and tracking.
- Washing batch receipt and processing.
- Delivery batches and delivery confirmation.
- Status history and auditability.
- Notifications and realtime status updates.
- Laundry history.
- Complaints and operational communication.
- RLS and server-side authorization.

4.2 Explicitly Out of MVP

- Payment processing and subscription billing.
- Route optimization.
- AI prediction.
- Advanced customer loyalty.
- Complex WhatsApp integration.
- Full-scale chat platform.
- Multi-vendor marketplace.
- Automated SMS infrastructure.
- Native Android/iOS applications.
- Advanced business intelligence.

5. Functional Requirements

ID	Requirement	Description
FR-01	Authentication	The system shall allow users to authenticate using supported Supabase Auth flows.
FR-02	Role Detection	The system shall determine the authenticated user's role and expose only the appropriate workflow.
FR-03	Protected Routes	The system shall prevent unauthorized users from accessing role-specific pages and operations.
FR-04	Student Profile	Students shall be able to view and manage permitted profile information.
FR-05	Pickup Announcements	Students shall be able to view pickup announcements relevant to their hostel/session.

ID	Requirement	Description
FR-06	Pickup Session	Authorized delivery/admin users shall be able to create, announce, start and manage pickup sessions.
FR-07	QR Collection	Authorized delivery staff shall be able to scan a bag QR and collect a valid active laundry bag.
FR-08	Order Creation	A valid collection shall create/associate a laundry order with the bag, owner and pickup context.
FR-09	Collection Progress	Delivery staff shall be able to see the number/status of bags collected in a session.
FR-10	Facility Receipt	Washing staff shall be able to receive the relevant pickup/batch at the washing facility.
FR-11	Washing Queue	Washing staff shall be able to view and process laundry orders in the washing queue.
FR-12	Washing Status	Authorized washing operations shall move orders through controlled washing states.
FR-13	Ready for Delivery	Completed washing shall make eligible orders ready for delivery.
FR-14	Delivery Batch	Authorized delivery staff shall be able to create/manage a delivery batch.
FR-15	Out for Delivery	Orders assigned to a delivery batch shall be transitioned to out-for-delivery through controlled operations.
FR-16	Delivery Confirmation	Delivery staff shall be able to scan/confirm a delivered bag.
FR-17	Laundry Timeline	Students shall be able to view the current status and historical status progression of their laundry.
FR-18	Laundry History	Completed laundry shall remain available in history and shall not be silently deleted.
FR-19	Notifications	The system shall create/display relevant pickup and laundry status notifications.
FR-20	Realtime Updates	Relevant status changes shall be reflected in the UI through realtime refresh/update mechanisms where configured.

ID	Requirement	Description
FR-21	Complaints	Students shall be able to report laundry problems and view their complaints.
FR-22	Communication	Students shall be able to communicate with relevant staff through the supported complaint/message workflow.
FR-23	Admin Management	Admins shall be able to manage users, bags, hostels, facilities, orders, batches and complaints within authorized operations.
FR-24	Auditability	Relevant operational actions shall be traceable through status history/audit records.

6. End-to-End Workflow

6.1 Pickup and Collection

- An authorized delivery/admin user schedules a pickup session for a hostel.
- Students receive/view the pickup announcement.
- The delivery boy starts the pickup session.
- A student brings the laundry bag to the collection point.
- The delivery boy scans the permanent QR on the bag.
- The server validates the identifier, bag, owner, hostel/session context, permissions and active-order conditions.
- A valid collection creates/updates the laundry order and records the COLLECTED state.
- The student can immediately see the updated laundry status.

6.2 Facility and Washing

- Collected laundry is associated with the operational washing batch.
- The batch is transported to the washing facility.
- Washing staff receive the relevant batch.
- Individual bags/orders can be identified/scanned within the washing workflow.
- Orders transition to RECEIVED_AT_FACILITY and then UNDER_WASHING through controlled operations.
- When processing is complete, the order transitions to READY_FOR_DELIVERY.

6.3 Delivery

- Delivery staff create or use an appropriate delivery batch.
- Ready orders are assigned to the delivery batch.
- The batch is moved to OUT_FOR_DELIVERY.
- The delivery boy reaches the hostel.
- The student presents/receives the bag and the delivery boy scans/validates it.
- The system confirms delivery and records DELIVERED.

• The student sees the completed laundry in history.

6.4 Core Operational Loop

ANNOUNCE ↓ COLLECT ↓ SCAN ↓ TRACK ↓ RECEIVE ↓ WASH ↓ READY ↓ DELIVER ↓ SCAN ↓ COMPLETE

7. Business Rules and State Management

7.1 Status Model

Status	Meaning
COLLECTED	Delivery staff has accepted the bag into the pickup workflow.
RECEIVED_AT_FACILITY	The laundry has reached the washing facility.
UNDER_WASHING	Washing/processing is in progress.
READY_FOR_DELIVERY	Processing is complete and the order can be delivered.
OUT_FOR_DELIVERY	The order has been assigned/taken for delivery.
DELIVERY_ATTEMPTED	A delivery attempt occurred but was not completed.
DELIVERED	The laundry has been successfully delivered.

7.2 Controlled Transitions

Laundry status transitions must be centralized and authorization-aware. Client-side code must not be allowed to arbitrarily mutate a laundry status. Database functions/RPCs, RLS policies and server-side logic are used to enforce valid operations.

7.3 Bag Identity

A bag has a permanent QR identity. The QR identifies the physical bag; it does not directly expose sensitive student information. A reusable bag can participate in multiple laundry orders over time.

7.4 Duplicate Prevention

- Duplicate QR scans must not create duplicate active laundry orders.
- Already delivered orders must be handled safely.
- Invalid or inactive bags must be rejected.
- A bag belonging to the wrong hostel/context must be rejected where the workflow requires hostel matching.

8. Data Requirements

The implemented project uses a relational PostgreSQL model in Supabase. The following are the principal business entities reflected by the current project and migrations.

Entity	Purpose / Key Relationship
profiles	Authenticated user's application profile, role and permitted organizational information.
hostels	Hostel/collection location context.
facilities	Washing/operational facility information.

Entity	Purpose / Key Relationship
bags	Permanent physical bag identity, including QR token and owner relationship.
pickup_sessions	Scheduled collection window and delivery-staff context.
laundry_orders	A single laundry transaction tied to a physical bag and student.
washing_batches	Group of laundry orders received/processed at the facility.
delivery_batches	Group of ready orders assigned for delivery.
status_history	Historical state changes for traceability.
notifications	User-facing operational/status messages.
complaints	Student-reported laundry problems.
messages	Supported communication associated with operational issues.
audit_logs	Traceability of relevant administrative/operational actions.

8.1 Database Design Principles

- Use relational constraints and foreign keys to preserve integrity.
- Use indexes for operational lookups and workflow performance.
- Use RLS to restrict records by role and ownership.
- Use database functions/RPCs for sensitive workflow transitions where appropriate.
- Preserve completed laundry history rather than deleting completed orders.
- Maintain status history for traceability.

9. External Interface Requirements

9.1 User Interface

- Responsive web UI for student, delivery, washing and admin roles.
- Reusable navigation, cards, status badges, timelines, tables, dialogs and forms.
- Loading, empty and error states for major features.
- Human-readable status labels instead of raw database enum values.

9.2 QR Scanner Interface

- Browser camera QR scanning.
- Repeated scanning without requiring unnecessary page reloads.
- Server-side identifier validation.
- Clear success/error feedback.
- Safe handling of camera permission denial, invalid QR and duplicate scans.

9.3 External Services

Service	Purpose
Supabase Auth	User authentication and sessions.
Supabase PostgreSQL	Persistent relational application data.
Supabase Realtime	Live status/operational refresh.
Vercel	Web application deployment.
Browser Camera / QR library	QR scanning and identification.

10. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	Sensitive operations must be authorized server-side and protected by RLS/role rules.
NFR-02	Traceability	Important operational state changes must be auditable.
NFR-03	Reliability	The application should fail safely on invalid identifiers, duplicate operations and network/realtime failures.
NFR-04	Usability	Student workflows should remain simple despite underlying system complexity.
NFR-05	Responsiveness	The web UI should be usable across common desktop and mobile viewport sizes.
NFR-06	Maintainability	Use reusable components, feature modules and domain services rather than duplicating logic.
NFR-07	Consistency	Status labels, validation and workflow rules should be centralized.
NFR-08	Performance	Operational lookups and dashboards should use appropriate indexes and efficient queries.
NFR-09	Scalability	Architecture should support growth in students, bags, orders and operational history.
NFR-10	Availability	Production deployment should use managed Vercel/Supabase services.

11. Security and Authorization

11.1 Authentication

Supabase Auth is the authentication provider. Sessions are integrated into the Next.js application through the Supabase client/server utilities and authentication middleware.

11.2 Authorization

- Role-based access must be enforced for Student, Delivery Boy, Washing Staff and Admin.
- Authorization must be enforced server-side; hiding a UI button is not sufficient.
- RLS policies must restrict access to permitted records.
- Sensitive database operations should be executed through protected server logic or database functions/RPCs.

11.3 QR Security

QR codes represent a bag identity and must not expose sensitive student information. The scanned identifier must be resolved and validated server-side against current workflow context.

11.4 Data Integrity

- Do not bypass centralized laundry status transitions.
- Do not allow unauthorized direct database mutations from client code.
- Do not delete completed laundry orders as part of normal workflow.

12. System Architecture

USER ■ ▼ ■■■■■■■■■■■■■■■■■■■■■■■■ ■ Next.js / React UI ■ ■ Pages & Components ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■ ▼ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ Server Actions / API Routes■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ▼ ■■■■■■■■■■■■■■■■■■■■■■■■ ■ Features / Services ■ ■ Business Operations ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■ ▼ ■■■■■■■■■■■■■■■■■■■■■■■■ ■ Supabase Client / ■ ■ Server Integration ■ ■■■■■■■■■■■■■■■■■■■■■■■■ ■ ▼ ■■■■■■■■■■■■■■■■■■■

13. Current Project Structure

The following classification reflects the current repository structure.

Path	Classification	Purpose
src/app/(student)/**	Frontend + server actions	Student dashboards, laundry, history, complaints, notifications and profile.
src/app/admin/**	Frontend + server actions	Admin dashboard and management workflows.
src/app/delivery/**	Frontend + server actions	Pickup, scanning, collection and delivery workflows.
src/app/washing/**	Frontend + server actions	Facility receipt, queue, washing and scanner workflows.
src/app/auth/**	Frontend + backend	Login, signup, password and auth callback flows.
src/app/api/**	Backend/API	HTTP route handlers such as current-user and Supabase health.
src/components/**	Frontend	Reusable UI components.
src/features/**	Backend/data access	Role-specific data retrieval and dashboard data.
src/lib/**	Backend/shared	Auth, Supabase integration, validation and utilities.
src/services/**	Backend/domain services	Core application operations.
src/types/**	Shared	TypeScript enums and result/types.
scripts/seed-local-test-data.mjs	Development tooling	Creates local test Auth/data.
supabase/migrations/**	Database	Schema, RLS, RPCs, business rules, seeds and realtime configuration.
supabase/seed.sql	Database seed entrypoint	Local seed configuration/entrypoint.
supabase/local-test-roster.*	Testing	Test roster and QR/login reference data.

14. Testing and Acceptance Criteria

14.1 Testing Levels

- Unit tests for isolated logic.
- Integration tests for services and database operations.
- Database/RLS tests for authorization and integrity.
- Workflow tests for collection, washing and delivery transitions.
- QR tests for valid, invalid, duplicate and unauthorized scans.
- End-to-end tests for the complete lifecycle.

14.2 QR Collection Acceptance

- Camera can remain active for repeated scans.

- QR token can be recognized.
- Identifier is validated server-side.
- Bag exists and is active.
- Owner exists and context is valid.
- No conflicting active order exists.
- Pickup session is active.
- Delivery staff is authorized.
- Order is created successfully.
- Status history and notification behavior occur as required.
- Duplicate and invalid scans are rejected safely.

14.3 Critical End-to-End Acceptance Test

- Delivery boy schedules pickup.
- Students receive pickup notification.
- Delivery boy starts pickup.
- Student submits bag.
- Delivery boy scans QR.
- System creates the laundry order.
- Student sees COLLECTED.
- Washing staff receives the bag.
- Student sees RECEIVED_AT_FACILITY.
- Washing starts.
- Student sees UNDER_WASHING.
- Washing completes.
- Student sees READY_FOR_DELIVERY.
- Delivery batch is created.
- Order becomes OUT_FOR_DELIVERY.
- Student receives delivery notification.
- Delivery boy scans the bag.
- Order becomes DELIVERED.
- Student sees the completed laundry in history.
- Complete status history remains preserved.

15. MVP Scope and Out of Scope

15.1 MVP

The production-capable MVP includes authentication, role-based access, student/delivery/washing/admin dashboards, bag and QR operations, pickup sessions, laundry orders, washing and delivery workflows, status history, notifications, history, complaints, audit logs, RLS and realtime status updates.

15.2 Deferred Features

Payment, subscriptions, route optimization, AI prediction, advanced loyalty, complex WhatsApp integration, full chat, marketplace functionality, automated SMS, native mobile applications and advanced BI are explicitly deferred unless

later requested.

16. Deployment and Environment Requirements

16.1 Local Development

Local development uses Next.js and a local Supabase development environment backed by Docker. Local test data is prepared by the project's test-data seed script.

16.2 Production

The intended production arrangement is Vercel for the Next.js web application and Supabase Cloud for authentication, PostgreSQL and realtime services.

16.3 Environment Variables

- NEXT_PUBLIC_SUPABASE_URL — Supabase project URL.
- NEXT_PUBLIC_SUPABASE_ANON_KEY / publishable key — browser-safe Supabase key as used by the application.
- SUPABASE_SERVICE_ROLE_KEY — privileged server-side key; must never be exposed to browser code.
- NEXT_PUBLIC_APP_URL — application base URL where required for auth redirects.

Secrets must be stored in local environment files or deployment environment-variable settings and must not be committed to source control.

17. Risks, Edge Cases and Failure Handling

- Duplicate QR scan.
- Invalid QR identifier.
- Inactive bag.
- Wrong hostel/context.
- Wrong delivery staff.
- Unauthorized washing staff.
- Already delivered order.
- Order missing from a batch.
- Delivery attempt / student unavailable.
- Damaged or missing laundry.
- Incorrect delivery confirmation.
- Duplicate active order.
- Cancelled pickup.
- Incomplete batch.
- Network interruption.
- Camera permission denied.
- Realtime connection failure.

The application should fail safely: reject invalid operations, preserve existing valid state, provide clear user feedback and avoid creating duplicate or inconsistent records.

18. Traceability and Definition of Done

18.1 Requirement-to-Implementation Traceability

Requirement Area	Primary Implementation Location
Authentication	src/app/auth/**, src/lib/auth/**, src/lib/supabase/**
Student workflow	src/app/(student)/**, src/features/student/**, src/services/student.service.ts
Delivery workflow	src/app/delivery/**, src/features/delivery/**, src/services/delivery.service.ts
Washing workflow	src/app/washing/**, src/features/washing/**, src/services/washing.service.ts
Admin workflow	src/app/admin/**, src/features/admin/**, src/services/admin.service.ts
Laundry domain	src/services/laundry.service.ts and database transitions
Pickup domain	src/services/pickup.service.ts and pickup-related migrations
Notifications	src/services/notification.service.ts and notification UI
Complaints	src/services/complaint.service.ts and complaint pages/forms
Database schema/security	supabase/migrations/**
QR scanning	src/components/qr-scanner.tsx, delivery/scan/**, washing/scan/**
Validation	src/lib/validation/schemas.ts

18.2 Definition of Done

- UI implemented.
- Loading state implemented.
- Empty state implemented.
- Error state implemented.
- Server-side validation implemented.
- Authorization implemented.
- Database operation implemented.
- Relevant audit/history behavior implemented.
- Notifications implemented where applicable.
- Realtime behavior implemented where applicable.
- Tests implemented.
- Relevant edge cases considered.

19. Conclusion

Aqua Track is a QR-based laundry tracking and workflow management platform designed to provide students with real-time visibility while giving operational staff a controlled digital workflow. The system is structured as a Next.js full-stack application with a clear presentation layer, server-side application logic and a Supabase/PostgreSQL persistence and security layer.

The critical success condition is an end-to-end, traceable journey from pickup announcement through QR collection, facility receipt, washing, ready-for-delivery, out-for-delivery and final delivery, with appropriate authorization, status history, notifications and safe failure handling at each step.

Appendix A — Key Project Directories

aqua-track/ ■■■ src/ ■ ■■■ app/ # Pages, layouts, server actions, API routes ■ ■■■ components/ # Reusable UI ■ ■■■ features/ # Feature-specific data access ■ ■■■ lib/ # Auth, Supabase, validation, utilities ■ ■■■ services/ # Domain/application services ■ ■■■ types/ # Shared TypeScript types ■■■ scripts/ # Development/seed tooling ■■■ supabase/ ■■■ migrations/ # PostgreSQL schema, RLS, RPCs, business rules ■■■ seed.sql # Seed entrypoint ■■■ test assets # Roster/QR references

Appendix B — Reference Sources

Primary source: Aqua Track Master Specification / Aqua Track Master Specification Updated. Supporting source: current Aqua Track repository directory structure supplied with the project.